```
static int bind_helper(ENGINE *e)
        {
        if(!ENGINE_set_id(e, engine_openssl_id)
                        || !ENGINE_set_name(e, engine_
openssl_name)
#ifndef TEST_ENG_OPENSSL_NO_ALGORITHMS
#ifndef OPENSSL_NO_RSA
                        || !ENGINE_set_RSA(e, RSA_get_
default_method())
#endif
#ifndef OPENSSL_NO_DSA
                        || !ENGINE_set_DSA(e, DSA_get_
default_method())
#endif
#ifndef OPENSSL_NO_ECDH
                        || !ENGINE_set_ECDH(e, ECDH_
OpenSSL())
#endif
#ifndef OPENSSL_NO_ECDSA
                        || !ENGINE_set_ECDSA(e, ECDSA_
OpenSSL())
#endif
#ifndef OPENSSL_NO_DH
                        || !ENGINE_set_DH(e, DH_get_default_
method())
#endif
                        || !ENGINE_set_RAND(e, RAND_SSLeay())
#ifdef TEST_ENG_OPENSSL_RC4
                        || !ENGINE_set_ciphers(e, openssl_
ciphers)
#endif
#ifdef TEST_ENG_OPENSSL_SHA
                        || !ENGINE_set_digests(e, openssl_
digests)
#endif
#endif
#ifdef TEST_ENG_OPENSSL_PKEY
                        || !ENGINE_set_load_privkey_
function(e, openssl_load_privkey)
#endif
                        )
                return 0;
        /* If we add errors to this ENGINE, ensure the error
handling is setup here */
        /* openssl_load_error_strings(); */
        return 1;
        }

static ENGINE *engine_openssl(void)
        {
        ENGINE *ret = ENGINE_new();
        if(!ret)
                return NULL;
        if(!bind_helper(ret))
                {
                ENGINE_free(ret);
                return NULL;
                }
        return ret;
        }
void ENGINE_load_openssl(void)
        {
        ENGINE *toadd = engine_openssl();
        if(!toadd) return;
        ENGINE_add(toadd);
        /* If the "add" worked, it gets a structural
reference. So either way,
         * we release our just-created reference. */
        ENGINE_free(toadd);
        ERR_clear_error();
        }
```

The good thing about the engine interface is that an engine need not be the only software and hardware implementation supported by OpenSSL by default, but can be one's own implementation. So, let us assume we have a software token implementation, which implements the RSA algorithm. Let us write a sample code that will register our implementation with the OpenSSL engine interface. Let's call our engine 'mytest'.

```
    if ( 0 == (ENGINE_set_id(eng, "mytest") &&
                ENGINE_set_name(eng, "MyTest OpenSSL Engine")
&&
                ENGINE_set_load_privkey_function(eng, l_
LoadMyTestPrivateKey) &&
                ENGINE_set_ciphers(eng, l_CiphersCb) &&
                ENGINE_set_default_ciphers(eng) &&
                ENGINE_set_digests(eng, l_DigestsCb) &&
                ENGINE_set_default_digests(eng) &&
                ENGINE_set_RSA(eng, l_EngineRsaMethod()) ) ) {
        LOG_AND_EXIT("There was an error registering mytest
engine ")
    }

    if ( 0 == ENGINE_add(eng) ) {
LOG_AND_EXIT("There was an error adding mytest engine ");
    }

    if ( 0 == ENGINE_init(eng) ) {
LOG_AND_EXIT("There was an error initing mytest engine ")
    }
```

Customising the SSL/TLS communication channel

The BIO interface is an I/O abstraction layer that allows different kinds of I/O mechanisms to be implemented to customise communication as per real world needs. This is basically needed for implementers who look at the TLS